Credora Finance - Credit Risk & Loan Approval Analytics Platform: Final Project Report

Sundram Sharma

October 2026

1. Summary

I built an end-to-end analytics and machine-learning platform for a synthetic retail-lending book (5,000 customers, 7,500 applications, 40,079 transactions): a validated relational database, ten analytical SQL queries, 24 EDA charts, leakage-aware features, a default-prediction model, a 0-100 customer risk score with Low/Medium/High bands, and two dashboards (Streamlit and a Power BI project).

The honest headline: the default model reaches a held-out ROC-AUC of 0.73 (95% CI 0.69-0.77), **below the PRD target of 0.80**, and recall at the chosen threshold is 0.66, **below the 0.70 target**. The score is useful as a second opinion, not as an automatic decision. A single feature (debt-to-income) already gives AUC 0.70.

Who it is for and how success is measured. Underwriters need a defensible second opinion per applicant; risk managers need portfolio trends and a watchlist; business heads need approval and volume views. Success measures used: PRD targets (AUC >= 0.80, recall >= 0.70), integrity checks pass, SQL results reconcile with independent pandas calculations, risk bands separate observed default rates, every score has ranked reasons.

Verification status. Tested on SQLite, not PostgreSQL; the Streamlit page and Power BI project were generated and checked by script but never rendered/opened; scikit-learn, XGBoost and SHAP were unavailable, so a NumPy re-implementation, gradient boosting and occlusion explanations were used (Section 9).

2. Data, validation and cleaning

Four tables joined on customer_id. Twenty-three automated checks cover keys, 1:1 and 1:N relations, orphans, value ranges and business rules. The raw data is tidy: no duplicates, no orphan keys, no nulls outside the two legitimately empty columns (decision_date for 173 applications under review, loan_default for the 4,097 non-disbursed). Observed approval rate of decided applications 46.4% and disbursed default rate 23.4% match the PRD.

One blocking issue was found: two e-mail addresses are each shared by two *different* customers (different dates of birth, cities, phones). The PRD schema declares email UNIQUE, so a PostgreSQL bulk load of the raw file would fail. The later customer's e-mail

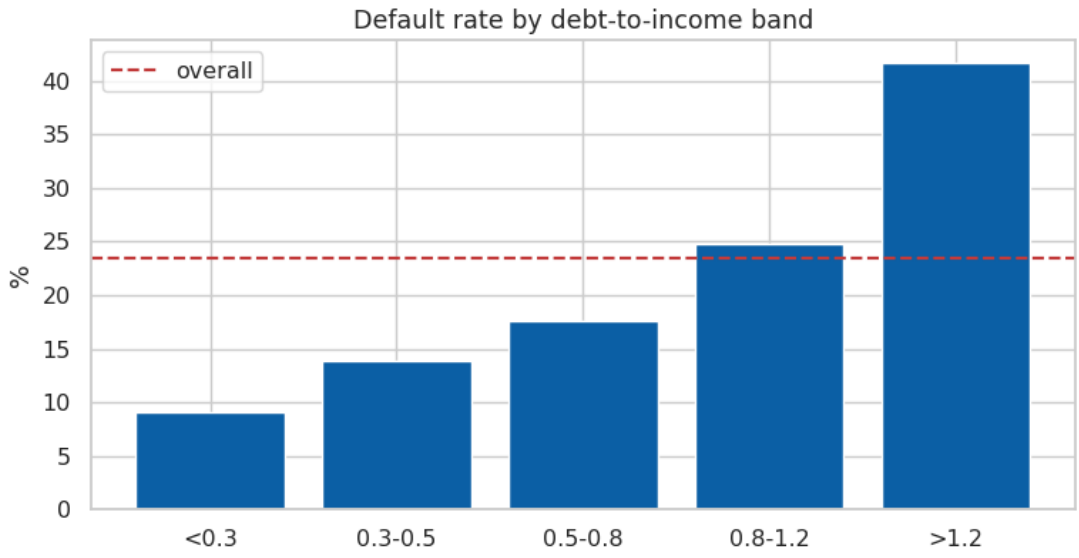
was set to NULL (e-mail is never used for modelling) and the load script uses the cleaned file. Other observations, documented but not changed: 935 applications are dated before customer_since; ages are computed as of mid-2026; the bureau snapshot is a single date (2024-06-30); transactions are sparse (4-12 per customer).

3. Database and SQL

The PostgreSQL schema (keys, CHECK constraints, ON DELETE CASCADE, indexes) comes from the PRD's SQL file. On SQLite with foreign keys on, invalid scores, orphan transactions and invalid statuses are all rejected. The ten analytical queries were run *from that file* after translating three PostgreSQL-only constructs; results match pandas (for example, Home-loan default rate 31.6%, credit_score<600 default rate 40.1%). Eight dashboard queries (one per PRD component) and the scored_customers DDL are in sql/scored_customers.sql. **Running the files on real PostgreSQL remains to be done.**

4. Exploratory analysis

- **Debt-to-income is the main driver.** Default rate by DTI band: <0.3: 9.1% | 0.3-0.5: 13.9% | 0.5-0.8: 17.6% | 0.8-1.2: 24.8% | >1.2: 41.7%. DTI>0.5 vs <=0.5: 28.5% vs 12.2% (z = 10.4).
- **Credit score is second.** Default rate by band: <580: 39.9% | 580-649: 34.8% | 650-699: 24.4% | 700-749: 16.8% | 750+: 11.5%.
- **Not everything is monotonic:** credit utilisation above 90% shows a *lower* default rate (16.7%) than 70-90% (36.8%).
- **Selection effect:** disbursed loans have median DTI 0.71, non-disbursed 2.39. Repayment is only observed for loans the business already judged safe.
- Geography and time effects are small (approval rate varies by about 8 points across states, 11 across months) and rest on small samples. 1,096 customers never applied; 55.9% of disbursed rows belong to customers with more than one loan.



Default rate by DTI band

5. Features

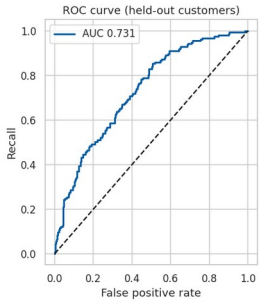
Application-grain (7,500 rows) and customer-grain (5,000 rows) tables. Transactions are aggregated to one row per customer *before* joining (no row fan-out, tested). Numeric inputs are median-imputed and winsorised at the training 1st/99th percentile because loan-to-income reaches 100x and inflow/outflow 169x. Excluded: post-decision fields, contact details and identifiers, gender and marital status (no fairness audit was run), city/state.

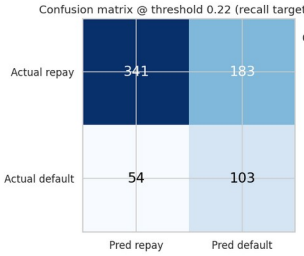
6. Default model

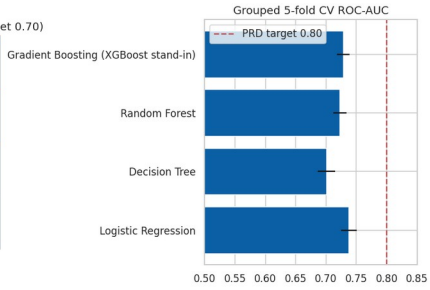
Trained on the 3,403 disbursed loans with a **customer-grouped** 80/20 split and grouped 5-fold CV for tuning (test set touched once).

Algorithm	Tuned setting	CV ROC-AUC	Test ROC-AUC
Logistic Regression	C=0.003	0.737	0.731
Decision Tree	depth=3, leaf=30	0.701	0.718
Random Forest	depth=8, leaf=10	0.723	0.720
Gradient Boosting (XGBoost stand-in)	n=100, depth=2	0.728	0.735

Logistic Regression, Gradient Boosting (standing in for XGBoost) and Random Forest are within about 0.015 of each other, so the simplest and most explainable model was selected. Probabilities are Platt-calibrated on out-of-fold predictions; thresholds were fixed on training folds. At threshold 0.224 the test set shows recall 0.66 and precision 0.36 (183 repaid loans flagged for 103 defaulters caught; 54 missed).







Evaluation

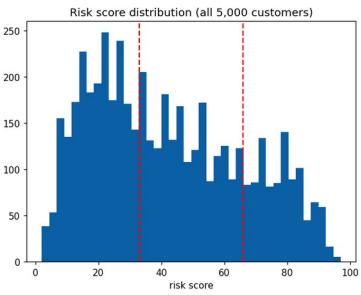
Is the number trustworthy? A naive random row split gives 0.755, so customer overlap would have flattered the result. Removing the bureau features (measured after most applications) gives AUC 0.702: the result does not rest on them, although temporal leakage cannot be excluded. With only 157 test defaults the AUC confidence interval is about ±0.04.

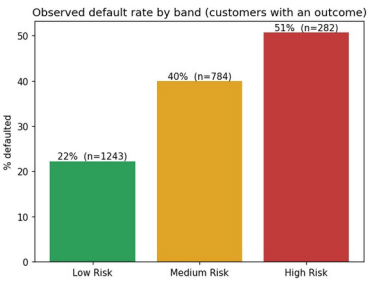
Drivers. Permutation importance: debt-to-income (AUC drop 0.051) far ahead of credit score (0.010), 30-day late payments, requested amount. The PRD expected utilisation, prior defaults and employment type to lead; they rank lower, partly because credit score already carries their signal (correlations of -0.31, -0.37 and -0.49 with utilisation, prior defaults and late payments).

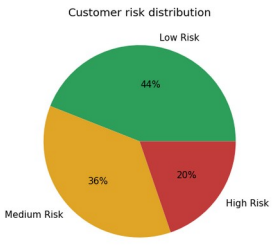
7. Risk score

risk_score = round(100 x calibrated P(default)); bands 0-33 Low, 34-66 Medium, 67-100 High (PRD defaults, kept because they separate outcomes). Customers who applied receive the highest probability over their applications; the 1,096 who never applied are scored by a weaker profile-only model (AUC about 0.66).

Band	Customers	Observed default rate (customers with an outcome)
Low	2,201	22.2%
Medium	1,811	39.9%
High	988	50.7%







Risk scoring

Every score carries its top three risk factors (occlusion method). **Caveat:** probabilities for non-disbursed applications average 0.53 versus 0.24 for disbursed loans; they are extrapolations of a model trained on safer loans and should be read as “high-risk profile”, not a measured default probability.

8. Dashboards

The Streamlit dashboard (dashboard/app.py) shows all eight PRD components with filters for loan type, state, risk category and month, plus an applicant lookup. The Power BI project has five pages covering the same components, a model-performance page and an honest AUC note. Credit-score “trend” is a cohort comparison by application month because bureau data is a single snapshot.

9. Testing and what was not verified

Area	Evidence
SQL layer	12 tests (constraints reject bad rows, queries match pandas)
Models, features, scores	18 tests (grouped splits, calibration, thresholds, no leaky/PII inputs, scores valid and explained)
Dashboard data layer	10 tests (SQL and service results agree)
Pipeline and notebooks	run_pipeline.py ran end to end; 5 notebooks executed with no errors
Not verified	PostgreSQL execution; rendering of the Streamlit page; opening the Power BI project; scikit-learn / XGBoost / SHAP results

10. Limitations and next steps

Selection bias (outcomes only for approved loans), temporal mismatch between bureau/transaction data and application dates, a small test set, sparse transactions, synthetic data, no fairness audit. Next: run on PostgreSQL; render and fix the dashboards; install the real libraries and re-run; run a bias review across gender, age and geography; study reject inference; validate on a later time window.